APUNTES COMPLETOS

El Modelo de Kuramoto

Sincronización, Metaestabilidad y Dinámica Cerebral

Proyecto de Sistemas Complejos – Universidad de Granada

Curso 2025-2026

Contents

I Fundamentos Teóricos **3**

1 Introducción Histórica y Motivación **3**
1.1 El Fenómeno de la Sincronización . . . 3
1.2 El Trabajo de Winfree y Kuramoto . . . 3

2 Derivación del Modelo de Kuramoto **4**
2.1 De Osciladores de Ciclo Límite a Osciladores de Fase . . . 4
2.2 Acoplamiento entre Osciladores . . . 4
2.3 El Modelo de Kuramoto Estándar . . . 4
2.4 Interpretación Física del Acoplamiento . . . 5

3 El Parámetro de Orden **5**
3.1 Definición y Significado Geométrico . . . 5
3.2 Reformulación de la Ecuación de Kuramoto . . . 6
3.3 Cálculo Práctico del Parámetro de Orden . . . 6

4 La Transición de Sincronización **7**
4.1 Análisis Cualitativo . . . 7
4.2 Solución de Kuramoto para el Estado Estacionario . . . 7
4.3 Ecuación de Autoconsistencia . . . 7
4.4 Valor Crítico del Acoplamiento . . . 7
4.5 Naturaleza de la Transición . . . 8

II El Modelo en Redes Complejas **8**

5 Extensión a Topologías Arbitrarias **8**
5.1 El Modelo de Kuramoto en Redes . . . 8
5.2 Parámetros de Orden Locales y Globales . . . 9
5.3 Efecto de la Topología . . . 9
5.4 El Papel del Laplaciano de la Red . . . 10

6 Redes Modulares y Sincronización Jerárquica **10**
6.1 Estructura Modular . . . 10
6.2 Sincronización en Redes Modulares . . . 10
6.3 Redes Jerárquicas . . . 11

7 Metaestabilidad **11**
7.1 Definición Rigurosa . . . 11
7.2 Cuantificación de la Metaestabilidad . . . 11
7.3 Origen de la Metaestabilidad en Redes Modulares . . . 11
7.4 Metaestabilidad vs Ruido . . . 12

III Aplicación al Conectoma y la Neurociencia **12**

8 El Conectoma Humano **12**
8.1 ¿Qué es el Conectoma? . . . 12
8.2 Modelado del Conectoma con Kuramoto . . . 12
8.3 Hallazgos Principales . . . 12

9 Relevancia para la Función Cerebral **13**
9.1 La Hipótesis del Cerebro Metaestable . . . 13
9.2 Evidencia Experimental . . . 13
9.3 Implicaciones Clínicas . . . 13

IV Implementación Numérica y Guía Práctica **14**

10 Algoritmos de Simulación **14**
10.1 Método de Euler . . . 14
10.2 Método de Runge-Kutta 4 (RK4) . . . 14
10.3 Implementación Vectorizada en Python . . . 14
10.4 Simulación Completa . . . 15

11 Análisis de Resultados **16**
11.1 Barrido en K: Diagrama de Fase . . . 16
11.2 Análisis de Redes Modulares . . . 16
11.3 Construcción de Redes Modulares . . . 17

12 Errores Comunes y Cómo Evitarlos **17**

13 Parámetros Recomendados para el Proyecto **18**
13.1 Tarea 1: Transición Clásica . . . 18
13.2 Tarea 2: Red Modular Simple . . . 18
13.3 Tarea 3: Red Jerárquica . . . 19
13.4 Tarea 4: Conectoma . . . 19

V Apéndices **19**

14 Derivación del Valor Crítico K_c **19**

15 Código Python Completo **20**

16 Glosario Completo **25**

Part I
Fundamentos Teóricos

1 Introducción Histórica y Motivación

1.1 El Fenómeno de la Sincronización

La sincronización es uno de los fenómenos más ubicuos en la naturaleza. Christiaan Huygens, en 1665, observó que dos relojes de péndulo colgados de la misma viga tendían a sincronizar sus oscilaciones — un fenómeno que él llamó "simpatía de los relojes". Desde entonces, se han documentado innumerables ejemplos de sincronización espontánea:

- **Biología**: El parpadeo sincronizado de luciérnagas en el sudeste asiático, donde miles de insectos emiten luz al unísono. Los ciclos menstruales de mujeres que conviven. Los ritmos circadianos.
- **Fisiología**: Las células del nodo sinoauricular del corazón, que se sincronizan para producir un latido coordinado. Las neuronas del núcleo supraquiasmático que regulan el reloj biológico.
- **Física**: Arrays de láseres que se sincronizan para producir luz coherente. Uniones Josephson superconductoras. Puentes que oscilan peligrosamente cuando los peatones sincronizan sus pasos (como el Millennium Bridge de Londres en 2000).
- **Sociología**: Aplausos que pasan de caóticos a rítmicos. Cánticos de multitudes en estadios.

La pregunta fundamental que conecta todos estos fenómenos es: **¿cómo emerge el orden colectivo a partir de interacciones locales entre unidades que, individualmente, tienen comportamientos diferentes?**

1.2 El Trabajo de Winfree y Kuramoto

En 1967, Arthur Winfree propuso el primer modelo matemático general para la sincronización de osciladores biológicos. Su modelo era complejo y difícil de analizar, pero capturaba la idea esencial: cada oscilador tiene una fase y una frecuencia natural, y la interacción entre osciladores depende de sus fases relativas.

En 1975, Yoshiki Kuramoto, un físico japonés, propuso una simplificación dramática del modelo de Winfree que mantenía la física esencial pero era matemáticamente tratable. La genialidad de Kuramoto fue elegir una forma específica para la función de interacción — el seno de la diferencia de fases — que permitía soluciones analíticas exactas en el límite de infinitos osciladores.

Cita de Kuramoto

"Mi objetivo era encontrar el modelo más simple que mostrara una transición de fase entre estados incoherentes y sincronizados. La elección del acoplamiento sinusoidal no fue arbitraria: es el primer término en la expansión de Fourier de cualquier función de acoplamiento periódica, y resulta ser suficiente para capturar la física esencial."

2 Derivación del Modelo de Kuramoto

2.1 De Osciladores de Ciclo Límite a Osciladores de Fase

Consideremos un sistema de N osciladores no lineales, cada uno descrito por variables de estado $\mathbf{x}_i \in \mathbb{R}^n$. En ausencia de acoplamiento, cada oscilador sigue su propia dinámica:

$$\dot{\mathbf{x}}_i = \mathbf{F}_i(\mathbf{x}_i) \tag{1}$$

Supongamos que cada oscilador tiene un **ciclo límite estable** — una órbita periódica cerrada hacia la cual el sistema es atraído desde condiciones iniciales cercanas. Sobre este ciclo límite, podemos parametrizar el estado del oscilador por una única variable: su **fase** $\theta_i \in [0, 2\pi)$.

Oscilador de Fase

Un oscilador de fase es un sistema cuyo estado está completamente determinado por un ángulo $\theta \in [0, 2\pi)$ que evoluciona en el tiempo. En ausencia de perturbaciones:

$$\dot{\theta} = \omega \tag{2}$$

donde ω es la **frecuencia natural** del oscilador (en radianes por unidad de tiempo).

La reducción de un oscilador de ciclo límite a un oscilador de fase es válida cuando:

1. El acoplamiento entre osciladores es **débil** (no saca al sistema del ciclo límite)
2. Las perturbaciones son **lentas** comparadas con el período de oscilación

Bajo estas condiciones, la teoría de promedios permite derivar ecuaciones de fase efectivas.

2.2 Acoplamiento entre Osciladores

Cuando acoplamos débilmente dos osciladores de fase, la interacción debe depender de sus fases relativas (por simetría rotacional). La forma más general es:

$$\dot{\theta}_i = \omega_i + \sum_{j=1}^{N} \Gamma_{ij}(\theta_j - \theta_i) \tag{3}$$

donde Γ_{ij} es una función 2π-periódica que describe cómo el oscilador j afecta al oscilador i.

Kuramoto hizo la **aproximación sinusoidal**: retuvo solo el primer armónico de la expansión de Fourier de Γ:

$$\Gamma_{ij}(\phi) \approx K_{ij} \sin(\phi) \tag{4}$$

Esta elección no es arbitraria. El término sinusoidal:

- Es antisimétrico: $\sin(-\phi) = -\sin(\phi)$, lo que garantiza que si i atrae a j, entonces j atrae igualmente a i.
- Se anula cuando las fases son iguales: no hay interacción si ya están sincronizados.
- Es máximo cuando la diferencia de fases es $\pm\pi/2$: el "tirón" es máximo a 90 grados de desfase.

2.3 El Modelo de Kuramoto Estándar

En su forma más simple, Kuramoto consideró N osciladores con acoplamiento **global** (todos con todos) y **uniforme** (misma fuerza de interacción):

Modelo de Kuramoto Clásico

$$\boxed{\dot{\theta}_i = \omega_i + \frac{K}{N}\sum_{j=1}^{N}\sin(\theta_j - \theta_i), \quad i = 1, \ldots, N} \tag{5}$$

donde:

- $\theta_i(t)$: fase del oscilador i en el tiempo t
- ω_i: frecuencia natural del oscilador i, extraída de una distribución $g(\omega)$
- $K \geq 0$: constante de acoplamiento (parámetro de control)
- El factor $1/N$ asegura que el modelo tiene un límite bien definido cuando $N \to \infty$

2.4 Interpretación Física del Acoplamiento

Analicemos el término de interacción $\sin(\theta_j - \theta_i)$:

- Si $\theta_j > \theta_i$ (el oscilador j está "adelantado"):
  - $\sin(\theta_j - \theta_i) > 0$
  - Contribución positiva a $\dot{\theta}_i \Rightarrow$ el oscilador i **acelera**
- Si $\theta_j < \theta_i$ (el oscilador j está "retrasado"):
  - $\sin(\theta_j - \theta_i) < 0$
  - Contribución negativa a $\dot{\theta}_i \Rightarrow$ el oscilador i **desacelera**

El efecto neto es que cada oscilador recibe un "empujón" de todos los demás, tendiendo a ajustar su velocidad hacia la del grupo. Los que van adelantados son frenados; los que van retrasados son acelerados.

Analogía Mecánica

Imagina N corredores en una pista circular, cada uno con su velocidad preferida ω_i. Están conectados por muelles elásticos. Un corredor que va adelantado siente un tirón hacia atrás de los demás; uno que va retrasado siente un tirón hacia adelante. Si los muelles son débiles (K pequeño), cada uno corre a su ritmo. Si son fuertes (K grande), tienden a correr juntos.

3 El Parámetro de Orden

3.1 Definición y Significado Geométrico

Para cuantificar el grado de sincronización, Kuramoto introdujo el **parámetro de orden complejo**:

Parámetro de Orden de Kuramoto

$$\boxed{z(t) = R(t)e^{i\Psi(t)} = \frac{1}{N}\sum_{j=1}^{N} e^{i\theta_j(t)}} \tag{6}$$

donde $R(t) \geq 0$ y $\Psi(t) \in [0, 2\pi)$ son, respectivamente, el módulo y el argumento del número complejo $z(t)$.

Interpretación geométrica: Cada oscilador se representa como un punto en el círculo unitario del plano complejo, en la posición $e^{i\theta_j}$. El parámetro de orden z es el **centroide** (centro de masa) de estos N puntos.

- $R = |z|$ mide cuán "agrupados" están los puntos:
  - $R = 0$: puntos uniformemente distribuidos (incoherencia total)
  - $R = 1$: todos los puntos coinciden (sincronización perfecta)
  - $0 < R < 1$: agrupamiento parcial
- $\Psi = \arg(z)$ es la fase promedio del grupo.

3.2 Reformulación de la Ecuación de Kuramoto

El parámetro de orden permite reescribir elegantemente la ecuación de Kuramoto. Multiplicando la definición de z por $e^{-i\theta_i}$:

$$Re^{i(\Psi-\theta_i)} = \frac{1}{N}\sum_{j=1}^{N} e^{i(\theta_j-\theta_i)} \tag{7}$$

Tomando la parte imaginaria:

$$R\sin(\Psi - \theta_i) = \frac{1}{N}\sum_{j=1}^{N} \sin(\theta_j - \theta_i) \tag{8}$$

Por lo tanto, la ecuación de Kuramoto se puede escribir como:

$$\boxed{\dot{\theta}_i = \omega_i + KR\sin(\Psi - \theta_i)} \tag{9}$$

Interpretación de Campo Medio

En esta formulación, cada oscilador no interactúa directamente con los demás, sino con un **campo medio** caracterizado por:

- Amplitud KR: fuerza efectiva del acoplamiento, proporcional al grado de coherencia
- Fase Ψ: dirección del "atractor" colectivo

Cada oscilador es atraído hacia la fase colectiva Ψ con una fuerza proporcional a KR.

Esta es una propiedad de **autoconsistencia**: el campo medio (R, Ψ) es producido por los osciladores, y a su vez determina su dinámica. La sincronización refuerza el campo medio, que a su vez facilita más sincronización — un mecanismo de retroalimentación positiva.

3.3 Cálculo Práctico del Parámetro de Orden

En una simulación numérica, $R(t)$ se calcula como:

$$R(t) = \left|\frac{1}{N}\sum_{j=1}^{N} e^{i\theta_j(t)}\right| = \frac{1}{N}\sqrt{\left(\sum_j \cos\theta_j\right)^2 + \left(\sum_j \sin\theta_j\right)^2} \tag{10}$$

Y la fase colectiva:

$$\Psi(t) = \arctan\left(\frac{\sum_j \sin\theta_j}{\sum_j \cos\theta_j}\right) \tag{11}$$

4 La Transición de Sincronización

4.1 Análisis Cualitativo

El comportamiento del sistema depende críticamente del balance entre dos efectos:

1. **Dispersión de frecuencias**: Los osciladores tienen frecuencias naturales diferentes ω_i, lo que tiende a desincronizarlos.
2. **Acoplamiento**: La interacción sinusoidal tiende a sincronizarlos.

- K **pequeño**: La dispersión domina. Cada oscilador rota aproximadamente a su frecuencia natural. Las fases se distribuyen de forma casi uniforme en el círculo $\Rightarrow R \approx 0$.
- K **grande**: El acoplamiento domina. Los osciladores se "enganchan" en fase y rotan juntos a una frecuencia común $\Rightarrow R \approx 1$.
- K **intermedio**: Algunos osciladores se sincronizan (los de frecuencia cercana a la media), mientras otros "derivan" $\Rightarrow 0 < R < 1$.

4.2 Solución de Kuramoto para el Estado Estacionario

Kuramoto buscó soluciones estacionarias donde R es constante y Ψ rota uniformemente. En un sistema de referencia que rota con Ψ, podemos fijar $\Psi = 0$.

En estado estacionario, la ecuación para el oscilador i es:

$$\dot{\theta}_i = \omega_i - KR\sin\theta_i = 0 \quad \text{(osciladores enganchados)} \tag{12}$$

Esto tiene solución solo si $|\omega_i| \leq KR$. Definimos:

- **Osciladores enganchados** ($|\omega_i| \leq KR$): Tienen fase fija $\theta_i^* = \arcsin(\omega_i/KR)$.
- **Osciladores a la deriva** ($|\omega_i| > KR$): Rotan con velocidad media no nula.

4.3 Ecuación de Autoconsistencia

La condición de autoconsistencia relaciona R con la distribución de frecuencias $g(\omega)$. En el límite $N \to \infty$:

Ecuación de Autoconsistencia de Kuramoto

$$R = KR\int_{-KR}^{KR} \cos^2\theta \cdot g(KR\sin\theta)\, d\theta \tag{13}$$

o equivalentemente:

$$1 = K\int_{-\pi/2}^{\pi/2} \cos^2\theta \cdot g(KR\sin\theta)\, d\theta \tag{14}$$

Esta ecuación determina R en función de K y la distribución $g(\omega)$.

4.4 Valor Crítico del Acoplamiento

Cerca del punto de transición, $R \ll 1$, y podemos expandir:

$$1 \approx K\int_{-\pi/2}^{\pi/2} \cos^2\theta \cdot g(0)\, d\theta = Kg(0)\cdot\frac{\pi}{2} \tag{15}$$

Por lo tanto:

$$\boxed{K_c = \frac{2}{\pi g(0)}} \tag{16}$$

Distribución Lorentziana

Para $g(\omega) = \frac{\gamma}{\pi(\omega^2+\gamma^2)}$ (Lorentziana con anchura γ):

- $g(0) = 1/(\pi\gamma)$
- $K_c = 2\gamma$

La solución exacta es $R = \sqrt{1 - K_c/K}$ para $K > K_c$.

Distribución Gaussiana

Para $g(\omega) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\omega^2/(2\sigma^2)}$ (Gaussiana con desviación σ):

- $g(0) = 1/(\sqrt{2\pi}\sigma)$
- $K_c = \sqrt{8/\pi} \cdot \sigma \approx 1.596\sigma$

4.5 Naturaleza de la Transición

La transición de sincronización es una **transición de fase de segundo orden** (continua). Cerca de K_c:

$$R \sim (K - K_c)^\beta, \quad \beta = \frac{1}{2} \tag{17}$$

Características:

- R crece continuamente desde cero
- No hay histéresis
- El exponente crítico $\beta = 1/2$ es el mismo que en la teoría de campo medio de transiciones de fase magnéticas

Caso de Distribuciones Bimodales

Si la distribución de frecuencias tiene dos picos (bimodal), la transición puede ser de **primer orden** (discontinua), con histéresis y coexistencia de estados.

Part II
El Modelo en Redes Complejas

5 Extensión a Topologías Arbitrarias

5.1 El Modelo de Kuramoto en Redes

En sistemas reales, las interacciones no son "todos con todos". El modelo generalizado es:

Modelo de Kuramoto en Redes

$$\boxed{\dot{\theta}_i = \omega_i + K \sum_{j=1}^{N} A_{ij} \sin(\theta_j - \theta_i)} \tag{18}$$

donde A_{ij} es la **matriz de adyacencia**:

- $A_{ij} = 1$ si existe conexión entre i y j
- $A_{ij} = 0$ si no existe conexión
- Para redes pesadas: $A_{ij} \in [0, \infty)$ representa la fuerza de la conexión

Ahora el "campo" que siente cada oscilador depende solo de sus vecinos:

$$\dot{\theta}_i = \omega_i + K \sum_{j \in \mathcal{N}_i} A_{ij} \sin(\theta_j - \theta_i) \tag{19}$$

donde $\mathcal{N}_i = \{j : A_{ij} \neq 0\}$ es el conjunto de vecinos de i.

5.2 Parámetros de Orden Locales y Globales

En redes no homogéneas, es útil definir parámetros de orden a diferentes escalas:

Parámetro de orden global:

$$R(t) = \left| \frac{1}{N} \sum_{j=1}^{N} e^{i\theta_j(t)} \right| \tag{20}$$

Parámetro de orden local (para un módulo o comunidad m):

$$r_m(t) = \left| \frac{1}{|m|} \sum_{j \in m} e^{i\theta_j(t)} \right| \tag{21}$$

Parámetro de orden de un nodo (coherencia con sus vecinos):

$$r_i(t) = \left| \frac{1}{k_i} \sum_{j \in \mathcal{N}_i} e^{i\theta_j(t)} \right| \tag{22}$$

donde $k_i = \sum_j A_{ij}$ es el grado del nodo i.

5.3 Efecto de la Topología

La estructura de la red afecta profundamente la dinámica de sincronización:

- **Grafos completos**: Recuperamos el modelo de campo medio. Transición de fase bien definida en K_c.
- **Redes aleatorias (Erdős-Rényi)**: Para grafos suficientemente densos, el comportamiento es similar al campo medio. Para grafos dispersos, la transición es más gradual.
- **Redes de mundo pequeño**: La presencia de atajos (conexiones de largo alcance) facilita la sincronización global.
- **Redes libres de escala**: Los hubs (nodos altamente conectados) actúan como "marcapasos" y sincronizan primero.
- **Redes modulares**: Los módulos pueden sincronizarse internamente antes de sincronizarse entre sí. Esto da lugar a regímenes intermedios ricos.
- **Redes jerárquicas**: La sincronización se despliega en múltiples etapas, a diferentes escalas.

5.4 El Papel del Laplaciano de la Red

La dinámica linealizada cerca del estado sincronizado está gobernada por el **Laplaciano del grafo**:

$$L_{ij} = \delta_{ij} k_i - A_{ij} \tag{23}$$

donde $k_i = \sum_j A_{ij}$ es el grado del nodo i.

El Laplaciano tiene autovalores $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_N$. El autovalor más pequeño no trivial, λ_2 (la **conectividad algebraica**), determina la estabilidad de la sincronización:

- λ_2 grande $\Rightarrow$ sincronización más estable
- λ_2 pequeño $\Rightarrow$ sincronización más frágil

6 Redes Modulares y Sincronización Jerárquica

6.1 Estructura Modular

Una red **modular** tiene grupos (módulos o comunidades) de nodos que están densamente conectados internamente pero escasamente conectados entre sí.

Modularidad

La modularidad Q de una partición de la red mide cuánto se desvía la estructura de conexiones de una distribución aleatoria:

$$Q = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j) \tag{24}$$

donde m es el número total de aristas, c_i es el módulo al que pertenece i, y δ es la delta de Kronecker.

- $Q \approx 0$: estructura aleatoria
- $Q > 0.3$: estructura modular significativa

6.2 Sincronización en Redes Modulares

En redes modulares, la sincronización puede ocurrir en etapas:

Etapa 1 (K pequeño): Incoherencia total. Ni R ni r_m son significativos.

Etapa 2 (K intermedio bajo): Los módulos individuales empiezan a sincronizarse internamente. $r_m \to 1$ para algunos módulos, pero R sigue bajo porque los módulos no están sincronizados entre sí.

Etapa 3 (K intermedio alto): Más módulos se sincronizan internamente. La coherencia inter-modular fluctúa $\Rightarrow$ metaestabilidad.

Etapa 4 (K grande): Sincronización global. $R \approx r_m \approx 1$.

Modelo de Dos Bloques

Considera dos módulos de M nodos cada uno, conectados por $m \ll M$ enlaces "puente".
Para K pequeño: ambos módulos incoherentes.
Para K intermedio: cada módulo se sincroniza internamente ($r_1, r_2 \to 1$), pero las fases de los módulos oscilan una respecto a la otra (como dos "super-osciladores" acoplados débilmente) $\Rightarrow$ R fluctúa.
Para K grande: los módulos se enganchan y $R \to 1$.

6.3 Redes Jerárquicas

Una red **jerárquica** tiene módulos dentro de módulos, en múltiples niveles. Ejemplos: la organización del cerebro, redes sociales, sistemas de transporte.

Características de la sincronización en redes jerárquicas:

- La sincronización se despliega "de abajo hacia arriba": primero a escalas pequeñas, luego a escalas mayores.
- Hay múltiples valores críticos de K, uno para cada nivel jerárquico.
- El régimen metaestable puede ser muy amplio.

7 Metaestabilidad

7.1 Definición Rigurosa

Metaestabilidad en Sincronización

Un sistema de osciladores acoplados está en un régimen **metaestable** cuando:

1. El parámetro de orden global $R(t)$ no converge a un valor fijo
2. El sistema explora diferentes configuraciones de sincronización parcial a lo largo del tiempo
3. Estas configuraciones son "quasi-estables": persisten durante tiempos largos antes de transitar a otras

7.2 Cuantificación de la Metaestabilidad

La medida estándar es la **desviación estándar temporal** del parámetro de orden:

$$\boxed{\sigma_R = \sqrt{\langle R^2 \rangle_t - \langle R \rangle_t^2}} \tag{25}$$

donde $\langle \cdot \rangle_t$ denota promedio temporal.

Interpretación:

- $\sigma_R \approx 0$: El sistema está en un estado estacionario (ya sea sincronizado o incoherente).
- σ_R máximo: El sistema fluctúa ampliamente entre diferentes estados $\Rightarrow$ **máxima metaestabilidad**.

7.3 Origen de la Metaestabilidad en Redes Modulares

La metaestabilidad emerge cuando:

1. Los módulos pueden sincronizarse internamente (conexiones intra-modulares fuertes)
2. Pero la sincronización inter-modular es inestable (conexiones inter-modulares débiles + diferencias de frecuencia entre módulos)

El sistema "visita" configuraciones donde diferentes subconjuntos de módulos están transitoriamente coordinados, pero ninguna configuración es estable a largo plazo.

7.4 Metaestabilidad vs Ruido

Es importante distinguir:

- **Fluctuaciones por ruido**: Presentes en sistemas finitos, decrecen como $1/\sqrt{N}$.
- **Metaestabilidad intrínseca**: Surge de la estructura del sistema, persiste incluso para N grande.

En redes jerárquicas modulares como el conectoma, la metaestabilidad es principalmente estructural, no un artefacto del tamaño finito.

Part III
Aplicación al Conectoma y la Neurociencia

8 El Conectoma Humano

8.1 ¿Qué es el Conectoma?

Conectoma

El **conectoma** es el mapa completo de las conexiones estructurales del cerebro. A escala macroscópica, representa las conexiones de materia blanca (axones) entre regiones corticales y subcorticales.

Características del conectoma humano:

- **Número de nodos**: ∼100-1000 regiones (dependiendo del atlas usado)
- **Modularidad**: $Q \approx 0.4 - 0.6$ (fuertemente modular)
- **Estructura jerárquica**: Módulos dentro de módulos a múltiples escalas
- **Hubs**: Regiones altamente conectadas que integran información
- **Eficiencia**: Alto clustering local + caminos cortos globales (mundo pequeño)

8.2 Modelado del Conectoma con Kuramoto

El paper de Villegas et al. (2015) estudió la dinámica de Kuramoto sobre el conectoma humano (998 nodos):

$$\dot{\theta}_i = \omega_i + K \sum_{j=1}^{N} W_{ij} \sin(\theta_j - \theta_i) \tag{26}$$

donde W_{ij} es la matriz de pesos del conectoma (derivada de tractografía).

8.3 Hallazgos Principales

1. Fase intermedia extendida: Entre la incoherencia ($K \ll K_c$) y la sincronización global ($K \gg K_c$), existe una amplia región de K donde el sistema muestra comportamiento complejo.

2. Sincronización frustrada: La combinación de:

- Heterogeneidad de frecuencias (ω_i diferentes)
- Cuellos de botella estructurales (pocas conexiones entre módulos)

- Estructura jerárquica

impide la sincronización jerárquica "limpia". En su lugar, la coherencia crece de forma no monótona con K.

3. Estados tipo chimera: Coexistencia de regiones sincronizadas y no sincronizadas, emergiendo puramente de la topología (sin necesidad de desfases o inhomogeneidades artificiales).

4. Metaestabilidad máxima: En el régimen intermedio, σ_R alcanza su máximo. El sistema explora un amplio repertorio de configuraciones de sincronización parcial.

9 Relevancia para la Función Cerebral

9.1 La Hipótesis del Cerebro Metaestable

Propuesta por Kelso, Deco, Kringelbach y otros:

Hipótesis de Operación Metaestable

El cerebro en reposo opera en un régimen de **máxima metaestabilidad**, lo que le confiere:

- **Flexibilidad**: Capacidad de transitar rápidamente entre diferentes estados funcionales
- **Repertorio dinámico**: Amplio conjunto de configuraciones disponibles para responder a estímulos
- **Balance integración-segregación**: Ni completamente integrado (pérdida de especialización) ni completamente segregado (pérdida de coordinación)

9.2 Evidencia Experimental

fMRI en reposo: Las correlaciones funcionales entre regiones cerebrales fluctúan en el tiempo. El patrón de "connectivity dynamics" es consistente con modelos de osciladores acoplados operando cerca de una transición.

MEG/EEG: Se observan "avalanchas neuronales" con distribuciones de tamaño y duración tipo ley de potencias, indicativo de criticalidad.

Estados alterados:

- Anestesia: Reduce la metaestabilidad (el sistema se "congela" en configuraciones fijas)
- Psicodélicos: Aumentan la entropía y la variabilidad dinámica
- Meditación: Modifica los patrones de metaestabilidad

9.3 Implicaciones Clínicas

Alteraciones de la metaestabilidad se han asociado con:

- **Esquizofrenia**: Desacoplamiento entre estructura y función
- **Autismo**: Hiper-conectividad local, hipo-conectividad global
- **Demencia**: Pérdida progresiva de la variabilidad dinámica
- **Epilepsia**: Transiciones patológicas a estados hiper-sincronizados

Part IV
Implementación Numérica y Guía Práctica

10 Algoritmos de Simulación

10.1 Método de Euler

El método más simple para integrar las ecuaciones de Kuramoto:

```
Para cada paso de tiempo n:
    Para cada oscilador i:
        suma = 0
        Para cada oscilador j:
            suma += A[i,j] * sin(theta[j,n] - theta[i,n])
        theta[i,n+1] = theta[i,n] + dt * (omega[i] + K * suma)
```

Ventajas: Simple, fácil de implementar. **Desventajas**: Baja precisión, puede requerir dt muy pequeño.

10.2 Método de Runge-Kutta 4 (RK4)

Más preciso que Euler:

```
def f(theta, omega, K, A):
    # Calcula d(theta)/dt
    coupling = K * np.sum(A * np.sin(theta - theta[:,np.newaxis]), axis=1)
    return omega + coupling

k1 = dt * f(theta, omega, K, A)
k2 = dt * f(theta + k1/2, omega, K, A)
k3 = dt * f(theta + k2/2, omega, K, A)
k4 = dt * f(theta + k3, omega, K, A)
theta_new = theta + (k1 + 2*k2 + 2*k3 + k4) / 6
```

10.3 Implementación Vectorizada en Python

Para eficiencia, evitar bucles explícitos:

```
import numpy as np

def kuramoto_step(theta, omega, K, A, dt):
    """Un paso de integración (Euler)."""
    # theta: array (N,)
    # A: matriz de adyacencia (N, N)

    # Diferencias de fase: theta_j - theta_i
    dtheta = theta[np.newaxis, :] - theta[:, np.newaxis]  # (N, N)

    # Término de acoplamiento
    coupling = K * np.sum(A * np.sin(dtheta), axis=1)  # (N,)

    # Actualizar fases
```

```
    theta_new = theta + dt * (omega + coupling)

    return theta_new

def order_parameter(theta):
    """Calcula R y Psi."""
    z = np.mean(np.exp(1j * theta))
    R = np.abs(z)
    Psi = np.angle(z)
    return R, Psi
```

10.4 Simulación Completa

```
def simulate_kuramoto(N, K, T, dt, omega=None, A=None, sigma_omega=1.0):
    """
    Simula el modelo de Kuramoto.

    Parámetros:
    -----------
    N : int - número de osciladores
    K : float - constante de acoplamiento
    T : float - tiempo total de simulación
    dt : float - paso temporal
    omega : array (N,) - frecuencias naturales (si None, genera aleatorias)
    A : array (N,N) - matriz de adyacencia (si None, usa all-to-all)
    sigma_omega : float - desviación estándar de frecuencias

    Retorna:
    --------
    t : array - tiempos
    theta : array (N, n_steps) - fases
    R : array (n_steps,) - parámetro de orden
    """
    n_steps = int(T / dt)

    # Inicialización
    if omega is None:
        omega = np.random.normal(0, sigma_omega, N)
    if A is None:
        A = np.ones((N, N)) / N  # all-to-all normalizado

    theta = np.random.uniform(0, 2*np.pi, N)  # fases iniciales

    # Arrays para guardar resultados
    theta_history = np.zeros((N, n_steps))
    R_history = np.zeros(n_steps)

    # Simulación
    for n in range(n_steps):
        theta_history[:, n] = theta
        R_history[n], _ = order_parameter(theta)
        theta = kuramoto_step(theta, omega, K, A, dt)
```

```
    t = np.arange(n_steps) * dt
    return t, theta_history, R_history
```

11 Análisis de Resultados

11.1 Barrido en K: Diagrama de Fase

```
K_values = np.linspace(0, 5, 50)
R_mean = []
R_std = []

for K in K_values:
    t, theta, R = simulate_kuramoto(N=500, K=K, T=200, dt=0.01)

    # Descartar transitorio (primeras 100 unidades de tiempo)
    R_steady = R[int(100/0.01):]

    R_mean.append(np.mean(R_steady))
    R_std.append(np.std(R_steady))

# Graficar
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(K_values, R_mean, 'b-', linewidth=2)
plt.xlabel('K')
plt.ylabel(r'$\langle R \rangle$')
plt.title('Parámetro de orden medio')

plt.subplot(1, 2, 2)
plt.plot(K_values, R_std, 'r-', linewidth=2)
plt.xlabel('K')
plt.ylabel(r'$\sigma_R$')
plt.title('Metaestabilidad')
```

11.2 Análisis de Redes Modulares

```
def modular_order_parameters(theta, modules):
    """
    Calcula parámetros de orden por módulo.

    modules: lista de listas con índices de nodos en cada módulo
    """
    r_modules = []
    for m in modules:
        theta_m = theta[m]
        z_m = np.mean(np.exp(1j * theta_m))
        r_modules.append(np.abs(z_m))
    return np.array(r_modules)
```

11.3 Construcción de Redes Modulares

```
import networkx as nx

def create_modular_network(n_modules, nodes_per_module, p_intra, p_inter):
    """
    Crea una red modular.

    p_intra: probabilidad de conexión dentro de módulos
    p_inter: probabilidad de conexión entre módulos
    """
    N = n_modules * nodes_per_module
    A = np.zeros((N, N))

    for m in range(n_modules):
        start = m * nodes_per_module
        end = (m + 1) * nodes_per_module

        # Conexiones intra-módulo
        for i in range(start, end):
            for j in range(i+1, end):
                if np.random.random() < p_intra:
                    A[i, j] = A[j, i] = 1

        # Conexiones inter-módulo
        for m2 in range(m+1, n_modules):
            start2 = m2 * nodes_per_module
            end2 = (m2 + 1) * nodes_per_module
            for i in range(start, end):
                for j in range(start2, end2):
                    if np.random.random() < p_inter:
                        A[i, j] = A[j, i] = 1

    return A
```

12 Errores Comunes y Cómo Evitarlos

Lista de Errores Frecuentes

1. **No descartar el transitorio**
   - El sistema necesita tiempo para alcanzar el estado estacionario o el régimen metaestable
   - Descartar al menos las primeras ∼50-100 unidades de tiempo
2. **Promediar insuficientemente**
   - Los resultados dependen de las condiciones iniciales y la realización de ω_i
   - Promediar sobre múltiples realizaciones (típicamente 10-50)
3. **Paso temporal demasiado grande**

- Si dt es muy grande, la integración es inestable
- Regla: $dt \ll 1/\max|\omega_i|$ y $dt \ll 1/(K \cdot k_{\max})$

4. **No normalizar K correctamente**
   - En redes no homogéneas, el "K efectivo" depende del grado
   - Considerar normalizar por el grado: $K_{ij} = K/k_i$ o similar
5. **Confundir sincronización de fase con frecuencia**
   - Sincronización de fase: $\theta_i - \theta_j \approx$ constante
   - Sincronización de frecuencia: $\dot{\theta}_i \approx \dot{\theta}_j$
   - Un sistema puede tener sincronización de frecuencia sin sincronización de fase
6. **Ignorar la finitud del sistema**
   - En sistemas finitos, $R > 0$ incluso sin sincronización verdadera
   - Para N osciladores independientes: $\langle R \rangle \sim 1/\sqrt{N}$

13 Parámetros Recomendados para el Proyecto

13.1 Tarea 1: Transición Clásica

- $N = 1000$ osciladores
- Red: all-to-all (grafo completo)
- Distribución de frecuencias: Gaussiana con $\sigma = 1$
- Barrido: $K \in [0, 4]$ con ∼50 puntos
- Tiempo de simulación: $T = 200$, descartar $T_{trans} = 50$
- Paso temporal: $dt = 0.01$
- Promediar sobre 10 realizaciones de ω_i

13.2 Tarea 2: Red Modular Simple

- $N = 300$ (3 módulos de 100 nodos)
- Conectividad intra-módulo: $p_{intra} = 0.3$
- Conectividad inter-módulo: $p_{inter} = 0.01$
- Frecuencias: cada módulo con media diferente, e.g., $\omega_m \in \{-0.5, 0, 0.5\}$
- Barrido: $K \in [0, 10]$
- Calcular: R, r_m para cada módulo, σ_R

13.3 Tarea 3: Red Jerárquica

- Estructura: 2 niveles (4 sub-módulos de 50 nodos, agrupados en 2 módulos)
- Conectividades: $p_{sub} = 0.4$, $p_{mod} = 0.1$, $p_{global} = 0.01$
- Analizar cómo $\langle R \rangle$ y σ_R responden a los diferentes niveles

13.4 Tarea 4: Conectoma

- Usar matriz de conectividad del paper de Villegas o similar
- Comparar con versión aleatorizada (preservando grados)
- Analizar la distribución de r_m para los módulos identificados

Part V
Apéndices

14 Derivación del Valor Crítico K_c

Partimos de la ecuación de autoconsistencia en el límite $N \to \infty$. En estado estacionario, con $\Psi = 0$:

Para osciladores enganchados ($|\omega| \leq KR$):

$$\theta^* = \arcsin\left(\frac{\omega}{KR}\right) \tag{27}$$

Para osciladores a la deriva ($|\omega| > KR$): contribuyen $\langle e^{i\theta} \rangle_t = 0$ al parámetro de orden (promedian a cero).

La contribución de los osciladores enganchados al parámetro de orden es:

$$R = \int_{-KR}^{KR} g(\omega) \cos\theta^*(\omega)\, d\omega \tag{28}$$

Usando $\cos\theta^* = \sqrt{1 - (\omega/KR)^2}$:

$$R = \int_{-KR}^{KR} g(\omega) \sqrt{1 - \left(\frac{\omega}{KR}\right)^2}\, d\omega \tag{29}$$

Cambio de variable $\omega = KR \sin\phi$:

$$R = KR \int_{-\pi/2}^{\pi/2} g(KR\sin\phi) \cos^2\phi\, d\phi \tag{30}$$

Para $R \to 0$ (cerca de la transición):

$$1 = K \int_{-\pi/2}^{\pi/2} g(0) \cos^2\phi\, d\phi = Kg(0) \cdot \frac{\pi}{2} \tag{31}$$

Por lo tanto:

$$\boxed{K_c = \frac{2}{\pi g(0)}} \tag{32}$$

15 Código Python Completo

```
"""
Simulación completa del modelo de Kuramoto
para el proyecto de Sistemas Complejos
"""

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

class KuramotoModel:
    """Clase para simular el modelo de Kuramoto."""

    def __init__(self, N, omega=None, sigma_omega=1.0, seed=None):
        """
        Inicializa el modelo.

        Parámetros:
        -----------
        N : int
            Número de osciladores
        omega : array-like, opcional
            Frecuencias naturales. Si None, se generan aleatoriamente.
        sigma_omega : float
            Desviación estándar de las frecuencias (si se generan)
        seed : int, opcional
            Semilla para reproducibilidad
        """
        if seed is not None:
            np.random.seed(seed)

        self.N = N

        if omega is None:
            self.omega = np.random.normal(0, sigma_omega, N)
        else:
            self.omega = np.array(omega)

        # Por defecto, red all-to-all normalizada
        self.A = np.ones((N, N)) / N
        np.fill_diagonal(self.A, 0)

    def set_adjacency(self, A):
        """Define la matriz de adyacencia."""
        self.A = np.array(A)

    def create_modular_network(self, modules, p_intra, p_inter):
        """
        Crea una red modular.

        modules : list
```

```
        Lista de tamaños de cada módulo
    p_intra : float
        Probabilidad de conexión intra-módulo
    p_inter : float
        Probabilidad de conexión inter-módulo
    """
    N = sum(modules)
    A = np.zeros((N, N))

    boundaries = np.cumsum([0] + modules)

    for m, (start, end) in enumerate(zip(boundaries[:-1], boundaries[1:])):
        # Intra-módulo
        for i in range(start, end):
            for j in range(i+1, end):
                if np.random.random() < p_intra:
                    A[i, j] = A[j, i] = 1

        # Inter-módulo
        for m2 in range(m+1, len(modules)):
            start2, end2 = boundaries[m2], boundaries[m2+1]
            for i in range(start, end):
                for j in range(start2, end2):
                    if np.random.random() < p_inter:
                        A[i, j] = A[j, i] = 1

    self.A = A
    self.modules = [list(range(s, e))
                    for s, e in zip(boundaries[:-1], boundaries[1:])]

def _derivative(self, theta, t, K):
    """Calcula dtheta/dt."""
    dtheta = theta[np.newaxis, :] - theta[:, np.newaxis]
    coupling = K * np.sum(self.A * np.sin(dtheta), axis=1)
    return self.omega + coupling

def simulate(self, K, T, dt=0.01, theta0=None):
    """
    Ejecuta la simulación.

    Parámetros:
    -----------
    K : float
        Constante de acoplamiento
    T : float
        Tiempo total de simulación
    dt : float
        Paso temporal para guardar resultados
    theta0 : array-like, opcional
        Fases iniciales
```

```
        Retorna:
        --------
        t : array
            Tiempos
        theta : array (N, len(t))
            Fases de cada oscilador
        R : array
            Parámetro de orden global
        """
        if theta0 is None:
            theta0 = np.random.uniform(0, 2*np.pi, self.N)

        t = np.arange(0, T, dt)
        theta = odeint(self._derivative, theta0, t, args=(K,))
        theta = theta.T  # (N, len(t))

        # Calcular parámetro de orden
        z = np.mean(np.exp(1j * theta), axis=0)
        R = np.abs(z)

        return t, theta, R

    def order_parameter(self, theta):
        """Calcula R y Psi para un conjunto de fases."""
        z = np.mean(np.exp(1j * theta))
        return np.abs(z), np.angle(z)

    def modular_order_parameters(self, theta):
        """Calcula parámetro de orden por módulo."""
        if not hasattr(self, 'modules'):
            raise ValueError("No se han definido módulos")

        r_m = []
        for m in self.modules:
            z_m = np.mean(np.exp(1j * theta[m]))
            r_m.append(np.abs(z_m))
        return np.array(r_m)

    def phase_diagram(self, K_values, T=200, T_trans=50, dt=0.01, n_trials=10):
        """
        Genera el diagrama de fase R vs K.

        Retorna:
        --------
        K_values : array
        R_mean : array
        R_std : array
        """
        R_mean = []
        R_std = []
```

```
        for K in K_values:
            R_trials = []

            for _ in range(n_trials):
                # Regenerar frecuencias para cada trial
                self.omega = np.random.normal(0, 1, self.N)

                t, theta, R = self.simulate(K, T, dt)

                # Descartar transitorio
                idx_trans = int(T_trans / dt)
                R_steady = R[idx_trans:]

                R_trials.append(np.mean(R_steady))

            R_mean.append(np.mean(R_trials))
            R_std.append(np.std(R_trials))

        return np.array(K_values), np.array(R_mean), np.array(R_std)

    def metastability_diagram(self, K_values, T=200, T_trans=50, dt=0.01):
        """
        Calcula la metaestabilidad (sigma_R) vs K.
        """
        sigma_R = []

        for K in K_values:
            t, theta, R = self.simulate(K, T, dt)
            idx_trans = int(T_trans / dt)
            R_steady = R[idx_trans:]
            sigma_R.append(np.std(R_steady))

        return np.array(K_values), np.array(sigma_R)


# ============== EJEMPLO DE USO ==============

if __name__ == "__main__":

    # Tarea 1: Transición clásica
    print("Simulando transición clásica...")

    model = KuramotoModel(N=500, sigma_omega=1.0, seed=42)
    K_values = np.linspace(0, 4, 40)

    K, R_mean, R_std = model.phase_diagram(K_values, T=150, T_trans=50, n_trials=5)
    _, sigma_R = model.metastability_diagram(K_values, T=150, T_trans=50)

    # Valor crítico teórico para Gaussiana
    K_c_theory = np.sqrt(8/np.pi) * 1.0  # sigma=1
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].errorbar(K, R_mean, yerr=R_std, fmt='o-', capsize=3)
axes[0].axvline(K_c_theory, color='r', linestyle='--',
                label=f'$K_c$ teórico = {K_c_theory:.2f}')
axes[0].set_xlabel('K')
axes[0].set_ylabel(r'$\langle R \rangle$')
axes[0].set_title('Transición de sincronización')
axes[0].legend()

axes[1].plot(K, sigma_R, 's-', color='green')
axes[1].set_xlabel('K')
axes[1].set_ylabel(r'$\sigma_R$')
axes[1].set_title('Metaestabilidad')

plt.tight_layout()
plt.savefig('kuramoto_transition.png', dpi=150)
plt.show()

# Tarea 2: Red modular
print("\nSimulando red modular...")

model2 = KuramotoModel(N=300)
model2.create_modular_network([100, 100, 100], p_intra=0.3, p_inter=0.02)

# Frecuencias diferentes por módulo
model2.omega[:100] = np.random.normal(-0.5, 0.3, 100)
model2.omega[100:200] = np.random.normal(0, 0.3, 100)
model2.omega[200:] = np.random.normal(0.5, 0.3, 100)

K_test = 3.0
t, theta, R = model2.simulate(K_test, T=100, dt=0.01)

# Calcular parámetros de orden por módulo
r_modules = np.zeros((3, len(t)))
for i, m in enumerate(model2.modules):
    z_m = np.mean(np.exp(1j * theta[m, :]), axis=0)
    r_modules[i, :] = np.abs(z_m)

fig, axes = plt.subplots(2, 1, figsize=(12, 8))

axes[0].plot(t, R, 'k-', linewidth=2, label='R global')
for i in range(3):
    axes[0].plot(t, r_modules[i, :], '--', alpha=0.7, label=f'r módulo {i+1}')
axes[0].set_xlabel('Tiempo')
axes[0].set_ylabel('Parámetro de orden')
axes[0].set_title(f'Sincronización en red modular (K={K_test})')
axes[0].legend()

# Visualizar red
import networkx as nx
```

```
G = nx.from_numpy_array(model2.A)
pos = nx.spring_layout(G, seed=42)
colors = ['red']*100 + ['blue']*100 + ['green']*100
axes[1].set_title('Estructura de la red modular')
nx.draw(G, pos, node_size=20, node_color=colors, ax=axes[1],
        edge_color='gray', alpha=0.5)

plt.tight_layout()
plt.savefig('kuramoto_modular.png', dpi=150)
plt.show()

print("Simulaciones completadas.")
```

16 Glosario Completo

Acoplamiento (K)
Parámetro que controla la intensidad de la interacción entre osciladores. Es el parámetro de control de la transición de sincronización.

Campo medio
Aproximación en la que cada oscilador interactúa con un "campo" promedio generado por todos los demás, en lugar de interactuar individualmente con cada uno.

Chimera
Estado donde coexisten regiones sincronizadas y no sincronizadas en un sistema espacialmente extendido.

Conectoma
Mapa completo de las conexiones estructurales (anatómicas) del cerebro.

Fase (θ)
Variable angular que describe el estado de un oscilador de ciclo límite. Evoluciona periódicamente en el intervalo $[0, 2\pi)$.

Frecuencia natural (ω)
Frecuencia intrínseca de un oscilador aislado, sin acoplamiento. Generalmente se extrae de una distribución $g(\omega)$.

Frustración
Situación donde las interacciones locales no pueden satisfacerse simultáneamente. En sincronización, ocurre cuando la estructura de la red y la heterogeneidad de frecuencias compiten.

Laplaciano del grafo (L)
Matriz $L = D - A$ donde D es la matriz diagonal de grados y A es la adyacencia. Sus autovalores determinan propiedades de sincronización.

Matriz de adyacencia (A)
Matriz que codifica las conexiones de una red. $A_{ij} = 1$ si los nodos i y j están conectados.

Metaestabilidad
Régimen donde el sistema fluctúa entre diferentes estados parcialmente sincronizados, sin establecerse en ninguno definitivamente.

Modularidad
Propiedad de una red de tener grupos (módulos) densamente conectados internamente pero escasamente entre sí.

Oscilador de ciclo límite
Sistema dinámico con una órbita periódica estable hacia la cual converge desde condiciones iniciales cercanas.

Oscilador de fase
Simplificación de un oscilador de ciclo límite donde el estado está descrito únicamente por la fase θ.

Parámetro de orden (R)
Medida de coherencia global. $R = |N^{-1} \sum_j e^{i\theta_j}|$. Va de 0 (incoherencia) a 1 (sincronización perfecta).

Parámetro de orden local (r_m)
Parámetro de orden calculado solo para los osciladores de un módulo o subconjunto.

Red jerárquica
Red con estructura modular a múltiples escalas: módulos dentro de módulos.

Sincronización
Estado donde los osciladores mantienen una relación de fase constante (o casi constante) en el tiempo.

Sincronización de frecuencia
Estado donde los osciladores tienen la misma frecuencia efectiva, aunque sus fases pueden diferir.

Transición de fase
Cambio cualitativo en el comportamiento de un sistema al variar un parámetro de control. En Kuramoto, ocurre en $K = K_c$.

Valor crítico (K_c)
Valor de K donde ocurre la transición de sincronización. Para distribuciones unimodales, $K_c = 2/(\pi g(0))$.

Fin de los apuntes

Proyecto de Sistemas Complejos
Universidad de Granada, 2026